

Aseguramiento de la Calidad del Software

Caracterización de Defectos





Contenido

- 1. Contexto y fundamentos de la calidad del software**
- 2. Estándar ISO/IEC 29110 para VSEs**
- 3. Defectos de software y su caracterización**
- 4. Clasificación, métricas y aplicación en ISO/IEC 29110**
- 5. Flujos de calidad e integración continua**
- 6. Caso de estudio aplicado**
- 7. Ejercicios prácticos**
- 8. Conclusiones**

¿Por qué importa la calidad del software?

Las aplicaciones de software son cada vez más críticas para las organizaciones, permitiéndoles cumplir sus tareas primordiales.

La región enfrenta una escasa cultura de pruebas formales: la mayoría de las PYMEs y EMPs no tienen procesos estructurados.

La falta de pruebas sistemáticas impacta directamente en tiempo, confiabilidad, usabilidad y costo del producto final.

Organizaciones como ISO desarrollan estándares (ISO/IEC 29110, ISO/IEC 9126) para impulsar buenas prácticas en empresas de todos los tamaños.

40-80%

del costo de
desarrollo se destina
a corregir defectos

Nivel 1

TMMI: la mayoría
de las organizaciones
en la región



Modelos y Estándares de Prueba de Software

Comparativa de los principales modelos de mejora de pruebas

Modelo	Publicación	Niveles	Gestión Riesgos	Aplicabilidad EMPs
TMM	1996	5 niveles	✓	Complejo para PYMEs
TMMI	2008	5 niveles	✓	Requiere adaptación
TPI	1998	N/A	✓	Solo mejora, no estructura
TMAP	1995	N/A	✓	Muy extenso para EMPs
ISO/IEC 29110	2011	Perfil básico	✓	Diseñado para EMPs

Fuente: Escobar-Sánchez & Fuertes-Díaz (2015); Muñoz et al. (2019)

¿En qué consiste el estándar ISO/IEC 29110?

¿Qué es ISO/IEC 29110?

Es un conjunto de estándares y guías de ingeniería de software diseñado específicamente para **Entidades Muy Pequeñas (VSEs)**, es decir, organizaciones con hasta 25 personas dedicadas al desarrollo de software.

Estructura del estándar:

- **Perfil Básico:** dos procesos principales — Gestión del Proyecto (GP) e Implementación del Software (IS)
- **Gestión del Proyecto:** planificación, seguimiento, control de cambios y cierre del proyecto
- **Implementación del Software:** análisis de requisitos, diseño, construcción, pruebas y entrega

Objetivo principal:

Proporcionar prácticas de ingeniería de software accesibles y proporcionadas, que permitan a las VSEs producir software de calidad con procesos disciplinados de **verificación, validación y mejora continua**.



Ventajas de la implementación del estándar ISO/IEC 29110

Para la organización:

- Mejora la capacidad de gestión de proyectos y control de costos
- Aumenta la competitividad frente a empresas de mayor tamaño
- Facilita el acceso a licitaciones y contratos que exigen certificación de calidad
- Proporciona un camino estructurado hacia estándares como ISO 9001 o CMMI

Para el proceso de desarrollo:

- Reduce el retrabajo mediante procesos disciplinados de verificación y validación
- Estandariza la documentación y trazabilidad de requisitos, diseño y pruebas
- Permite detectar y caracterizar defectos de forma sistemática, alimentando la mejora continua

Para el producto:

- Mayor confiabilidad y menor densidad de defectos en el software entregado
- Incrementa la satisfacción del cliente al cumplir con las especificaciones

¿Qué es un defecto en software?



Idea central: Un defecto es cualquier desviación entre el comportamiento esperado y el comportamiento real del software.

Según el modelo de pruebas funcionales (Escobar & Fuertes):

“La prueba funcional debe mostrar errores y disconformidades con la especificación”

Relación con ISO/IEC 29110:

- El estándar exige gestión disciplinada de verificación y validación
- La caracterización de defectos permite medir calidad, controlar riesgos y mejorar procesos

Modelo de Madurez de Pruebas Integrado (TMMi)

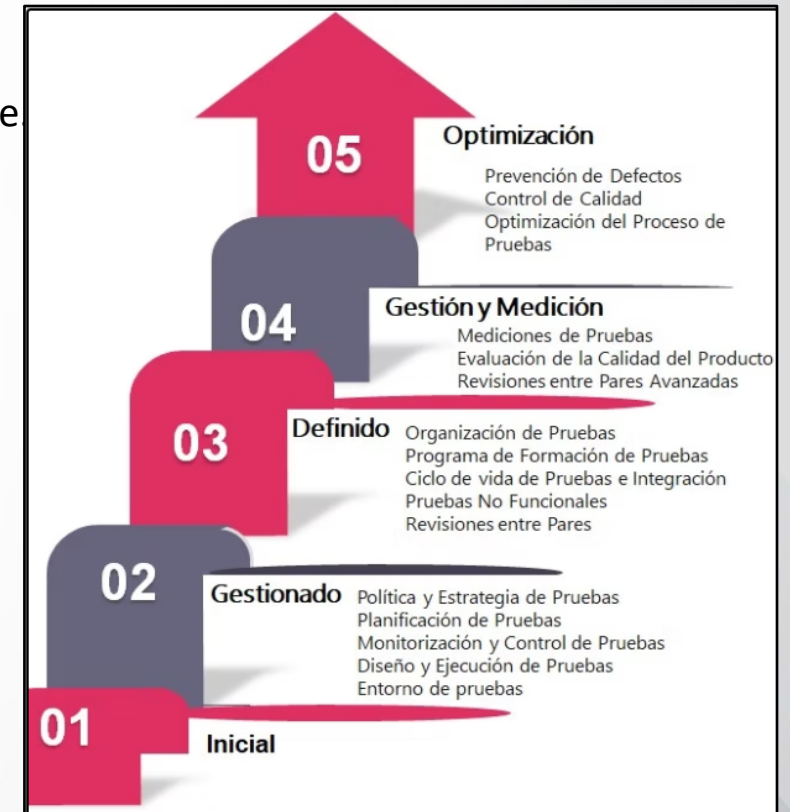
¿Qué es TMMi?

Es un modelo de referencia que evalúa y mejora la **madurez del proceso de pruebas** de software. Complementa a CMMI centrándose exclusivamente en las actividades de *testing*.

Los 5 niveles de madurez:

- **Nivel 1 – Inicial:** pruebas caóticas, sin proceso definido ni distinción entre pruebas y depuración
- **Nivel 2 – Gestionado:** política de pruebas, planificación, seguimiento y diseño basado en riesgo
- **Nivel 3 – Definido:** proceso estándar de pruebas integrado en el ciclo de vida del software
- **Nivel 4 – Medido:** métricas cuantitativas para evaluar calidad y efectividad de las pruebas
- **Nivel 5 – Optimizado:** mejora continua, prevención de defectos y automatización avanzada

TMMi Nivel 2 es el punto de partida para organizaciones que buscan formalizar sus pruebas y se alinea directamente con ISO/IEC 29110.



Importancia de caracterizar defectos

¿Por qué es crítico en ISO/IEC 29110 y TMMi Nivel 2?

- Permite planificar pruebas basadas en riesgo (TMMi Nivel 2)
- Facilita la mejora continua (ISO 9001 + ISO 29110)
- Reduce retrabajo y costos
- Aumenta la confiabilidad del producto

Métricas que alimenta:

- Densidad de defectos • Tasa de detección • Severidad promedio • Tendencia de reincidencia

Clasificación por severidad

“Son incidentes de baja importancia, que se reducen significativamente al realizar la segunda versión del producto.”

- **Crítico:** detiene el sistema o afecta funciones esenciales
- **Alto:** afecta funcionalidades principales sin detener el sistema
- **Medio:** afecta funciones secundarias
- **Bajo:** problemas cosméticos o de presentación

Clasificación por origen

- **Requisitos:** ambigüedad, omisiones, contradicciones
- **Diseño:** arquitectura incorrecta, flujos mal definidos
- **Codificación:** errores sintácticos, lógica incorrecta
- **Integración:** reprocesos por cambios concurrentes, fallos en despliegues
- **Pruebas:** casos mal diseñados, cobertura insuficiente

“Cuando varios desarrolladores trabajan sobre la misma sección del código... se enfrentan a reprocesos y fallos en los despliegues.”



Métricas asociadas a defectos (ISO/IEC 29110)

- **Densidad de defectos:** Defectos / KLOC o Defectos / Funcionalidad
- **Tasa de detección:** Defectos encontrados por ciclo de prueba
- **Tasa de corrección:** Tiempo promedio para resolver un defecto
- **Reincidencia:** Porcentaje de defectos reabiertos
- **Distribución por severidad:** Permite priorizar esfuerzos de mejora



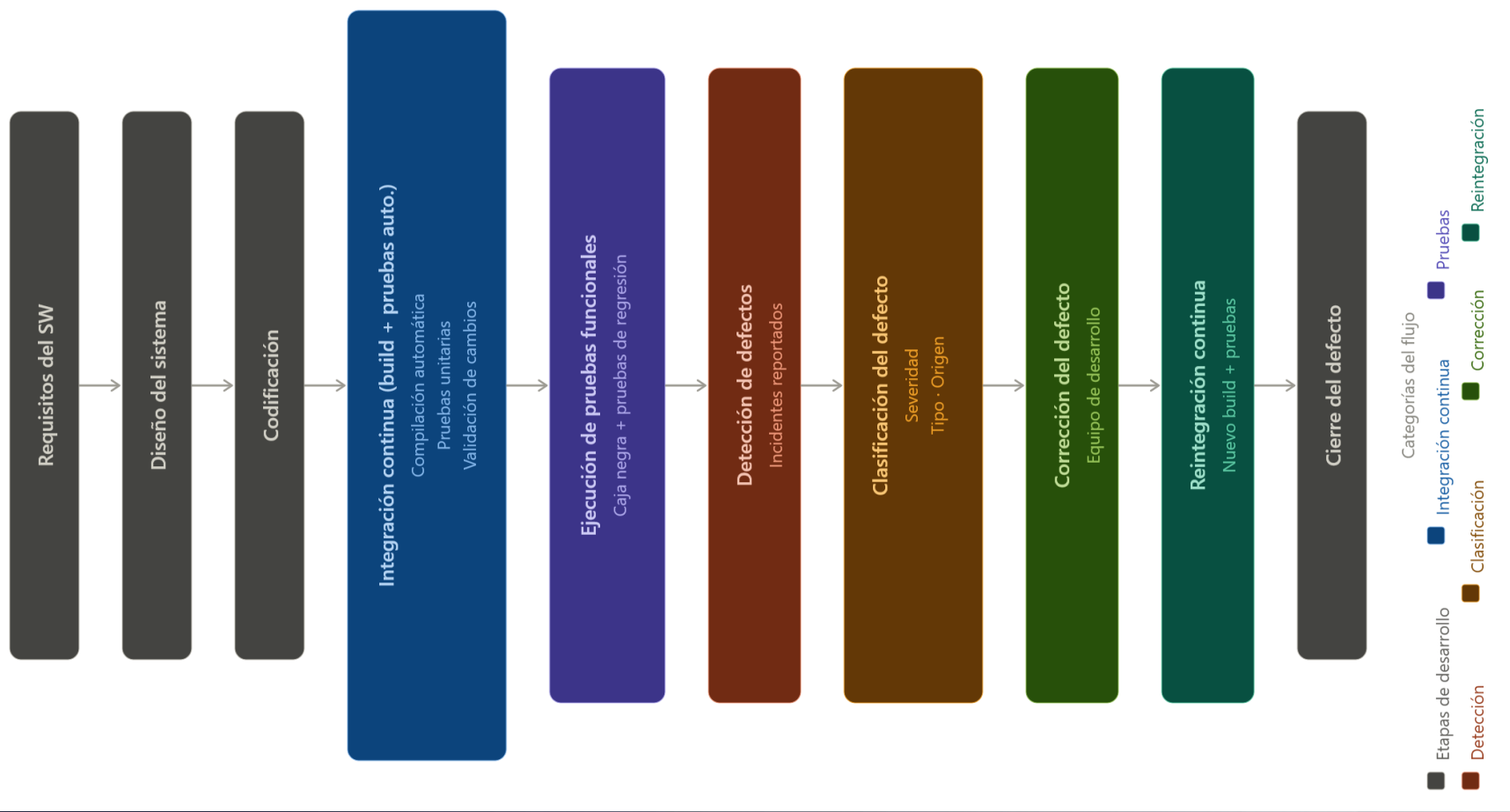
Cómo ISO/IEC 29110 usa esta caracterización

El estándar exige:

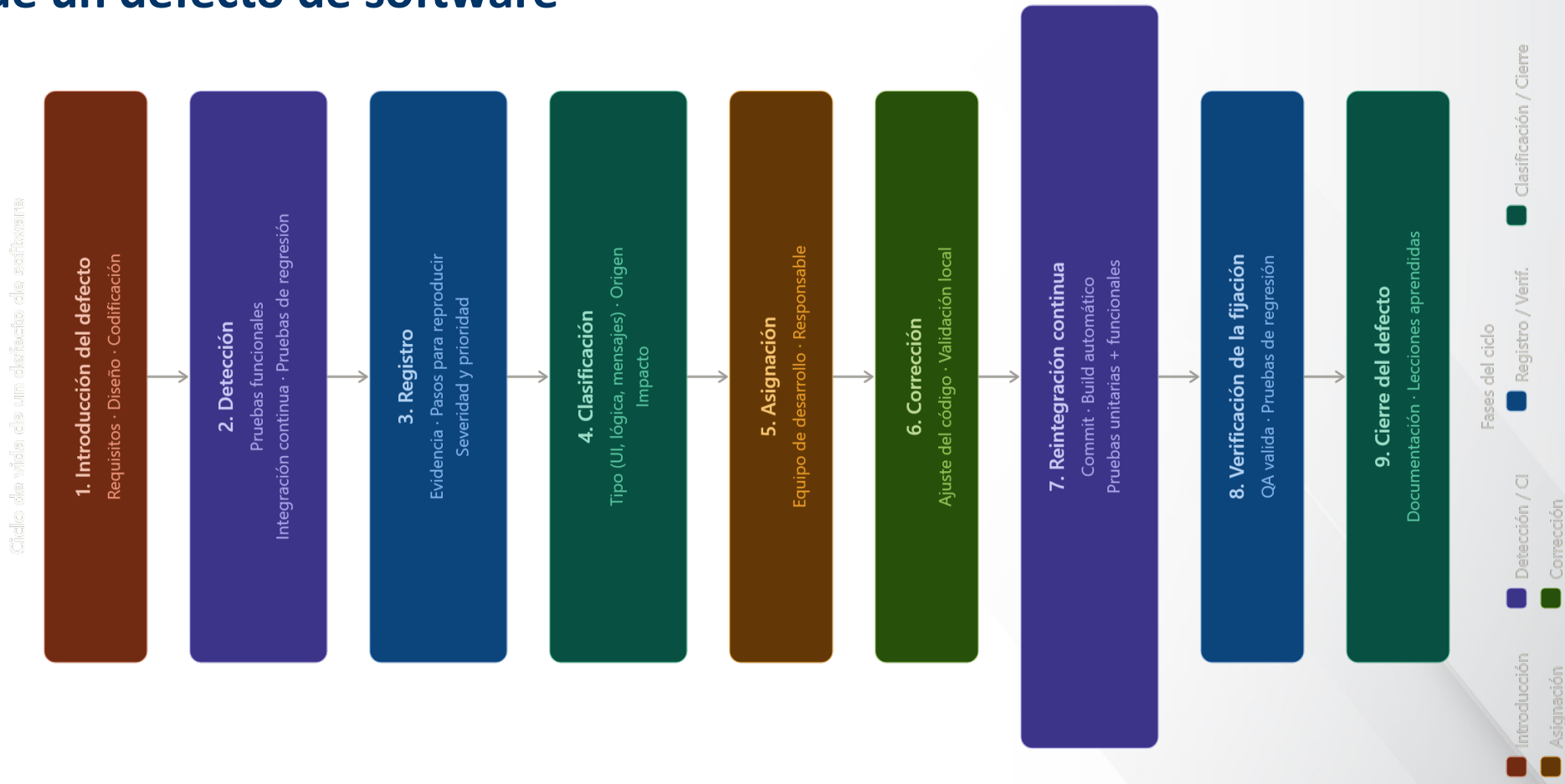
- Gestión disciplinada de verificación y validación
- Registro y análisis de no conformidades
- Control de cambios y correcciones
- Retroalimentación para mejora continua

La caracterización de defectos es el insumo principal para cumplir estos requisitos.

Flujo de Defectos en el Proceso de Aseguramiento de la Calidad



Ciclo de vida de un defecto de software



Defectos y su relación con Integración Continua

*“Compilar y validar automáticamente los cambios...
notificando inmediatamente las fallencias encontradas.”*



Beneficios de la IC:

- Detecta defectos temprano en el ciclo de desarrollo.
- Evita regresiones en el código.
- Asegura que la “main line” siempre esté estable.
- Automatiza pruebas unitarias y de integración.
- Reduce defectos por integración tardía.

Caso de estudio

Alcance de la especificación de la prueba, acordando con el cliente 20 funcionalidades de las 40 especificadas al inicio:

ALCANCE DE LA ESPECIFICACIÓN DE LA PRUEBA

Prioridad	Complejidad	Criticidad	Número de funcionalidades
Alta	Alta	Alta	5
Alta	Alta	Alta	5
Alta	Media	Media	5
Media	Media	Media	5
		Total	20

Ejemplo de funcionalidades:

1. Prioridad Alta – Complejidad Alta – Criticidad Alta (5 funcionalidades)

Estas funcionalidades son **núcleo del negocio**, complejas y críticas.

1. Autenticación *multifactor* (MFA) para usuarios administrativos
2. Procesamiento de pagos electrónicos con validación bancaria en tiempo real
3. Generación automática de facturación electrónica DIAN
4. Módulo de gestión de inventario con actualización en tiempo real
5. Motor de reglas de negocio para aprobación automática de solicitudes

2. Prioridad Alta – Complejidad Alta – Criticidad Media (5 funcionalidades)

Son importantes y complejas, pero no detienen el sistema si fallan.

1. *Dashboard* analítico con indicadores clave (KPI)
2. Integración con servicios externos vía API REST
3. Gestión avanzada de roles y permisos personalizados
4. Programación automática de tareas (cron interno)
5. Módulo de auditoría de actividades del usuario

3. Prioridad Alta – Complejidad Media – Criticidad Media (5 funcionalidades)

Importantes, de complejidad moderada y criticidad media.

1. Gestión de usuarios (crear, editar, desactivar)
2. Búsqueda avanzada con filtros dinámicos
3. Exportación de reportes en PDF y Excel
4. Notificaciones por correo electrónico
5. Historial de cambios por registro

4. Prioridad Media – Complejidad Media – Criticidad Media (5 funcionalidades)

Funcionalidades útiles, no críticas, de complejidad moderada.

1. Personalización del perfil del usuario
2. Configuración de preferencias del sistema (tema, idioma)
3. Gestión de categorías o etiquetas
4. Visualización de estadísticas básicas
5. Soporte para adjuntar archivos en formularios

Caracterización de Defectos de Software

Taxonomía según el modelo propuesto (TMMI Nivel 2)

 Funcionalidad Incorrecta	 Interfaz de Usuario	 Mensajes Erróneos o faltantes	 Defectos de Regresión
El módulo no implementa la especificación acordada con el cliente. Lógica mal implementada. Reglas de negocio incompletas. Cálculos erróneos.	Defectos de interfaz. Mensajes confusos, campos incorrectos, flujos no intuitivos. Controles mal ubicados. Inconsistencias visuales. Problemas de usabilidad.	Mensajes de error o informativos ausentes, incorrectos o inapropiados. Falta de validaciones. Mensajes ambiguos. Ausencia de retroalimentación al usuario	Cambios al código generan nuevos fallos en funcionalidades previas.

En el caso de estudio: Ciclo 1 → 59 incidentes | Ciclo 2 → 21 incidentes | Regresión → 20 incidentes (Total: 100)

Ejemplo real del documento

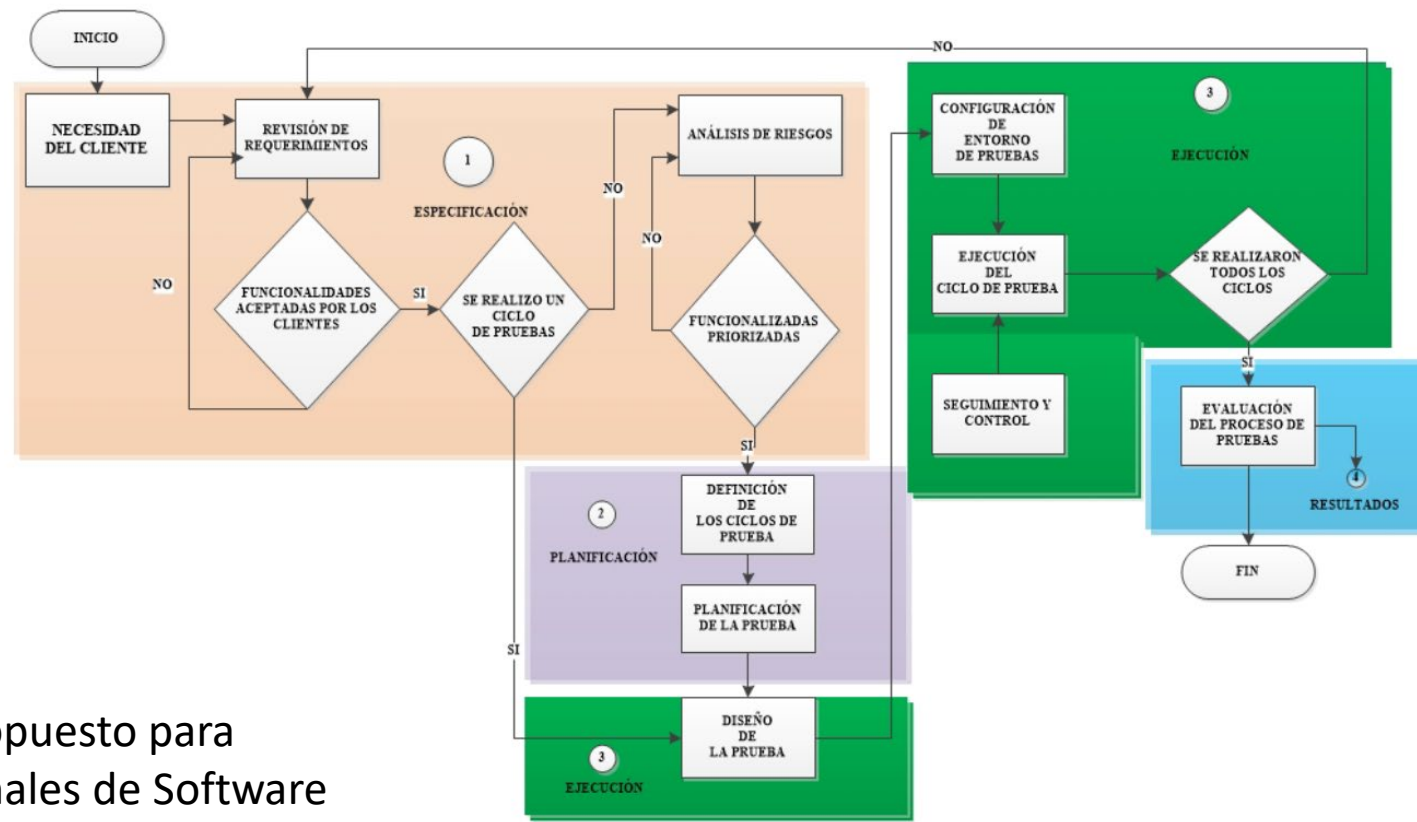
Datos del caso aplicado:

- **Ciclo 1:** 59 incidentes
- **Ciclo 2:** 21 incidentes
- **Regresión:** 20 incidentes
- **Total:** 100 incidentes

Interpretación didáctica:

- La reducción progresiva evidencia madurez del proceso
- La regresión confirma la importancia de automatizar pruebas
- La mayoría eran de mensajes y UI, no de lógica crítica

Modelo formal de pruebas funcionales de software para alcanzar el nivel de madurez integrado 2



Modelo propuesto para
Pruebas Funcionales de Software

EJERCICIOS PRÁCTICOS

Aplicación de la caracterización de defectos

Basado en: Escobar-Sánchez & Fuertes-Díaz (2015) · Muñoz, Peralta & Laporte (2019)



EJ. 1

Clasificar y priorizar defectos reales

Una EMP de 4 personas está desarrollando un sistema de facturación. El equipo de pruebas detectó los siguientes 6 incidentes en el Ciclo 1: (A) El botón “Guardar” no responde al hacer clic. (B) El logo aparece pixelado en pantallas HD. (C) El cálculo del IVA es incorrecto cuando hay descuentos. (D) El mensaje de error al ingresar un NIT inválido no aparece. (E) El menú de navegación se desordena en móvil. (F) El sistema no guarda facturas si hay más de 50 ítems.

¿LO QUE DEBES HACER:

- 1 Clasifica cada incidente según su TIPO (implementación, interfaz, mensaje, regresión).
- 2 Asigna un nivel de SEVERIDAD (crítico / alto / medio / bajo) a cada uno.
- 3 Identifica el ORIGEN probable de cada defecto (requisitos, diseño, codificación, pruebas).
- 4 Ordena los 6 defectos por prioridad de corrección y justifica tu criterio.

 **Entregable:** Tabla de clasificación con: Incidente | Tipo | Severidad | Origen | Prioridad | Justificación

RESPUESTA

Ejercicio 1 – Clasificación esperada

Incidente	Tipo	Severidad	Origen	Prioridad / Razón
(A) Botón no responde	Implementación	CRÍTICO	Codificación	Nº 1 – Bloquea guardar factura
(F) Falla >50 ítems	Implementación	CRÍTICO	Requisitos	Nº 2 – Impide uso en casos reales
(C) IVA incorrecto	Implementación	Alto	Codificación	Nº 3 – Error de negocio grave
(D) Mensaje NIT inv.	Mensaje	Alto	Diseño	Nº 4 – Afecta validación usuario
(E) Menú móvil	Interfaz	Medio	Diseño	Nº 5 – Usabilidad afectada
(B) Logo pixelado	Interfaz	Bajo	Diseño	Nº 6 – Sólo cosmético

EJ. 2

Calcular métricas y proponer plan de acción

La misma EMP cerró el Ciclo 2 con los siguientes datos: 8 funcionalidades probadas • 51 casos de prueba ejecutados • 21 incidentes detectados • Distribución por severidad: 3 críticos, 4 altos, 10 medios, 4 bajos. El Ciclo 1 tuvo 20 funcionalidades, 134 casos y 59 incidentes.

¿ LO QUE DEBES HACER:

- 1 Calcula la DENSIDAD DE DEFECTOS por funcionalidad en cada ciclo (Defectos / Funcionalidad).
- 2 Calcula la TASA DE DETECCIÓN por caso ejecutado en cada ciclo (Defectos / Casos).
- 3 Analiza la TENDENCIA: ¿el proceso está mejorando? Argumenta con los números.
- 4 Redacta un PLAN DE ACCIÓN de 3 medidas concretas para reducir los defectos críticos/altos en el Ciclo 3.

 **Entregable:** Hoja de cálculo con métricas + párrafo de análisis + plan de acción (máx. 15 líneas)

RESPUESTA

Ejercicio 2 – Métricas y plan de acción

Métrica	Ciclo 1	Ciclo 2	Tendencia
Funcionalidades	20	8	—
Casos ejecutados	134	51	—
Incidentes	59	21	↓ 64%
Densidad (def/func)	2.95	2.63	↓ Mejora
Tasa detección (def/caso)	0.44	0.41	↓ Mejora

⚙ Plan de acción sugerido para Ciclo 3:

1. Realizar revisión de código (code review) antes de ejecutar pruebas, enfocada en lógica de negocio.
2. Automatizar pruebas de regresión con casos críticos del Ciclo 1 y 2 para detectar recurrencias.
3. Aplicar técnica de partición de equivalencia en validaciones de entrada para reducir mensajes erróneos.



Conclusiones

1. Caracterizar defectos es fundamental para el aseguramiento de la calidad.
2. Permite medir, controlar y mejorar el proceso de desarrollo.
3. Se integra naturalmente con TMMi Nivel 2 y con ISO/IEC 29110.
4. La integración continua reduce defectos y mejora la estabilidad del producto.
5. La evidencia empírica muestra que la caracterización permite:
 - ✓ reduce incidentes,
 - ✓ prioriza esfuerzos y
 - ✓ aumenta satisfacción del cliente.

Aseguramiento de la Calidad del Software

Caracterización de Defectos

